

# 哈爾濱工業大學

## 模式识别与机器学习实验报告

实验 六

|   |       |                   |
|---|-------|-------------------|
| 题 | 目     | 基于生成模型的图像生成与编辑    |
| 学 | 院     | 计算学部              |
| 专 | 业     | 工科试验班(AI 先进技术领军班) |
| 学 | 号     | 2024113212        |
| 学 | 生     | 张泽晰               |
| 任 | 课 教 师 | 刘扬                |

哈尔滨工业大学计算机科学与技术学院

2021 年春季

目录

1 实验内容 3

2 实验环境 3

3 方法原理 3

3.1 VAE 模型思想 3

3.2 网络结构 4

4 关键代码 4

5 实验设计 6

6 实验结果及分析 7

6.1 最终损失对比 7

6.2 代表性图像结果 8

7 结论 8

8 参考文献 8

A 完整实验结果截图 9

A.1 数据量 10% 9

A.2 数据量 30% 9

A.3 数据量 100% 10

A.4 不使用 Dropout 11

A.5 使用 Dropout 12

A.6 隐变量维度 10 13

A.7 隐变量维度 20 14

A.8 隐变量维度 50 15

A.9 ReLU 激活函数 16

A.10 Leaky ReLU 激活函数 17

A.11 ELU 激活函数 18

## 1 实验内容

本实验围绕生成模型中的变分自编码器（Variational Autoencoder, VAE）展开。实验要求基于 PyTorch 在手写体数字图像数据集上训练生成模型，使模型能够从随机隐变量生成新的数字图像，并通过调整隐变量观察生成图像的连续变化，从而实现简单的图像编辑。

实验还要求比较不同数据量、不同超参数以及训练技巧对生成效果的影响。因此本报告在固定主体网络结构的基础上，分别考察训练数据比例、Dropout、隐变量维度和激活函数选择对最终损失、重建质量、随机生成质量以及隐变量编辑效果的影响。

## 2 实验环境

表 1 实验环境与主要配置

| 项目            | 配置                            |
|---------------|-------------------------------|
| 开发语言          | Python                        |
| 深度学习框架        | PyTorch、torchvision           |
| 数据集           | MNIST，训练集 60000 张，测试集 10000 张 |
| 优化器           | Adam                          |
| 学习率           | $1 \times 10^{-3}$            |
| 训练轮数          | 20 epoch                      |
| 默认 batch size | 128                           |
| 默认隐变量维度       | 20                            |
| 默认激活函数        | ReLU                          |
| 默认 Dropout    | 不使用                           |

## 3 方法原理

### 3.1 VAE 模型思想

VAE 将输入图像  $x$  编码为隐变量分布  $q_\phi(z|x)$ ，而不是一个确定的向量。编码器输出均值  $\mu$  与对数方差  $\log \sigma^2$ ，再通过重参数化技巧

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

得到可反向传播的隐变量采样。解码器学习从  $z$  还原图像，从而在训练结束后可以直接从标准正态分布中采样  $z$  生成新图像。

本实验采用的损失函数由重建误差与 KL 散度组成：

$$\mathcal{L} = \text{BCE}(x, \hat{x}) + \text{KL}(q_{\phi}(z|x)||p(z)) = \text{BCE}(x, \hat{x}) - \frac{1}{2} \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)$$

其中 BCE 约束重建图像接近原图，KL 散度约束隐变量分布接近标准正态分布，使潜空间具备连续、可采样的性质。

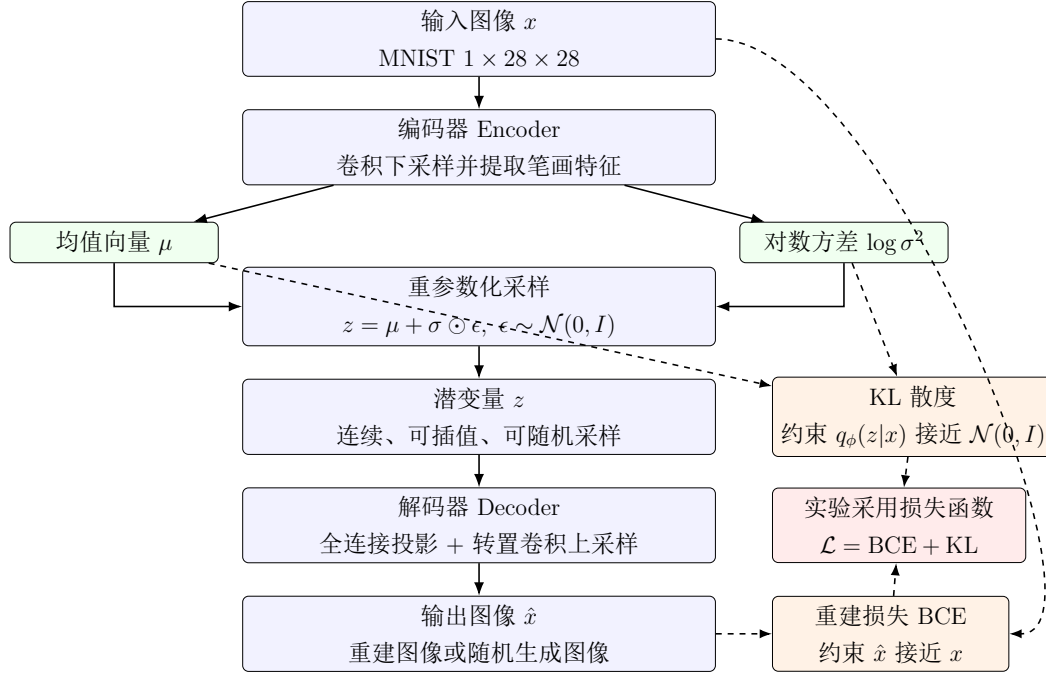


图 1 VAE 从编码、重参数化到解码生成的竖向流程图

### 3.2 网络结构

编码器使用三层卷积逐步下采样，将  $1 \times 28 \times 28$  的 MNIST 图像映射到高维特征并分别输出  $\mu$  与  $\log \sigma^2$ 。解码器先用全连接层将隐变量投影回特征图，再通过转置卷积逐步恢复空间分辨率，最终用 Sigmoid 输出灰度图像。

## 4 关键代码

报告仅列出与模型结构、重参数化、损失函数和实验配置直接相关的核心代码。

Listing 1 VAE 关键实现代码

```

1 class Encoder(nn.Module):
2     def __init__(self, latent_dim, use_dropout, dropout_rate, activation_fn):
3         super(Encoder, self).__init__()
4         self.act = activation_fn
5         self.conv1 = nn.Conv2d(1, 32, kernel_size=4, stride=2, padding=1)
6         self.conv2 = nn.Conv2d(32, 64, kernel_size=4, stride=2, padding=1)
7         self.conv3 = nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1)

```

```

8         self.fc_mu = nn.Linear(128 * 4 * 4, latent_dim)
9         self.fc_logvar = nn.Linear(128 * 4 * 4, latent_dim)
10        self.use_dropout = use_dropout
11        if use_dropout:
12            self.dropout = nn.Dropout2d(dropout_rate)
13
14        def forward(self, x):
15            x = self.act(self.conv1(x))
16            if self.use_dropout:
17                x = self.dropout(x)
18            x = self.act(self.conv2(x))
19            if self.use_dropout:
20                x = self.dropout(x)
21            x = self.act(self.conv3(x))
22            if self.use_dropout:
23                x = self.dropout(x)
24            x = x.view(x.size(0), -1)
25            mu = self.fc_mu(x)
26            logvar = self.fc_logvar(x)
27            return mu, logvar
28
29        # ...
30
31    class VAE(nn.Module):
32        def __init__(self, latent_dim, use_dropout, dropout_rate, activation_fn):
33            super(VAE, self).__init__()
34            self.encoder = Encoder(latent_dim, use_dropout, dropout_rate,
35                                   activation_fn)
36            self.decoder = Decoder(latent_dim, use_dropout, dropout_rate,
37                                   activation_fn)
38
39        def reparameterize(self, mu, logvar):
40            std = torch.exp(0.5 * logvar)
41            eps = torch.randn_like(std)
42            return mu + eps * std
43
44        def forward(self, x):
45            mu, logvar = self.encoder(x)
46            z = self.reparameterize(mu, logvar)
47            recon = self.decoder(z)
48            return recon, mu, logvar
49
50        # ...
51
52    def loss_function(recon_x, x, mu, logvar):
53        BCE = F.binary_cross_entropy(recon_x.reshape(-1, 784), x.reshape(-1, 784),
54                                      reduction='sum')
55        KLD = -0.5 * torch.sum(1 + logvar - mu.pow(2) - logvar.exp())
56        return BCE + KLD

```

```
54
55 # ...
56
57 EXPERIMENTS = [
58     # 1. 不同数据量
59     {'name': 'data_10%', 'data_fraction': 0.1},
60     {'name': 'data_30%', 'data_fraction': 0.3},
61     {'name': 'data_100%', 'data_fraction': 1.0},
62
63     # 2. Dropout 对比
64     {'name': 'dropout_False', 'use_dropout': False},
65     {'name': 'dropout_True', 'use_dropout': True},
66
67     # 3. 隐变量维度对比
68     {'name': 'latent_10', 'latent_dim': 10},
69     {'name': 'latent_20', 'latent_dim': 20},
70     {'name': 'latent_50', 'latent_dim': 50},
71
72     # 4. 激活函数对比
73     {'name': 'act_relu', 'activation': 'relu'},
74     {'name': 'act_leaky_relu', 'activation': 'leaky_relu'},
75     {'name': 'act_elu', 'activation': 'elu'},
76 ]
```

## 5 实验设计

实验采用控制变量法。除当前考察因素外，其余参数均保持默认设置。具体实验组如下：

- 数据量对比：使用 10%、30%、100% 的 MNIST 训练数据。
- Dropout 对比：比较不使用 Dropout 与使用 Dropout（概率 0.2）的效果。
- 隐变量维度对比：设置 latent dim 为 10、20、50。
- 激活函数对比：比较 ReLU、Leaky ReLU 与 ELU。

每个实验组均保存训练损失曲线、测试样本重建图、随机生成图以及沿隐变量若干维度线性插值得到的编辑结果。隐变量编辑时，将基础隐变量设为零向量，并在指定维度上从 -2.5 到 2.5 线性变化，观察生成图像的形态变化。

## 6 实验结果及分析

### 6.1 最终损失对比

表 2 各实验组最终训练损失

| 实验组            | Final Loss |
|----------------|------------|
| data_10%       | 104.9866   |
| data_30%       | 100.4926   |
| data_100%      | 97.2831    |
| dropout_False  | 97.2026    |
| dropout_True   | 106.1087   |
| latent_10      | 99.7648    |
| latent_20      | 97.4149    |
| latent_50      | 97.7283    |
| act_relu       | 97.5193    |
| act_leaky_relu | 98.1183    |
| act_elu        | 98.6805    |

从数据量对比看，训练数据比例由 10% 增加到 100% 时，最终损失由 104.9866 降低到 97.2831，说明更多样本能够提高生成模型对数字形态分布的刻画能力。10% 数据下模型已经可以学习到基本笔画结构，但重建与生成图像通常更模糊，泛化也更弱。

Dropout 对比中，不使用 Dropout 的最终损失为 97.2026，使用 Dropout 后为 106.1087。在本实验规模和网络结构下，Dropout 反而削弱了重建能力。这是因为 VAE 本身已经通过 KL 项引入正则化，额外的 Dropout 会破坏编码器和解码器中的细粒度笔画信息，使模型更难恢复 MNIST 图像。

隐变量维度对比中，latent dim 为 20 时损失最低，为 97.4149；维度 10 的表达能力略不足，而维度 50 并未带来明显收益。说明潜空间容量过小会限制数字形态表达，过大则可能增加优化难度或引入冗余维度，本实验中 20 维是较均衡的选择。

激活函数对比中，ReLU 的最终损失为 97.5193，略优于 Leaky ReLU 和 ELU。三者差距不大，表明该卷积 VAE 对激活函数不特别敏感，但 ReLU 在 MNIST 这类稀疏灰度图像上具备简单稳定的优化表现。

## 6.2 代表性图像结果

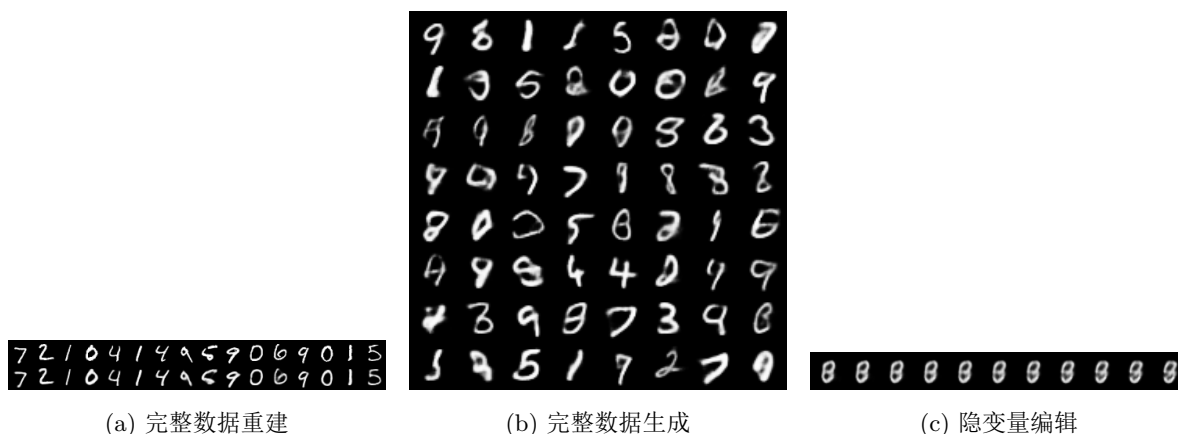


图 2 默认优选配置附近的代表性结果

重建图中，上半部分为原始测试样本，下半部分为对应重建结果。可以看到模型能够较好保留数字类别和主要笔画，但对于较细的笔画粗细、倾斜角和局部空洞仍存在平滑化现象，这是 VAE 使用概率潜空间和 BCE 重建目标时常见的结果。随机生成图显示模型已经学习到手写数字的整体分布，能够生成多种数字形态；隐变量编辑结果则说明潜空间具备连续性，沿单一维度移动时，图像亮度、粗细或局部结构会发生渐变。

## 7 结论

本实验基于 PyTorch 实现了卷积 VAE，并在 MNIST 数据集上完成了图像重建、随机生成和隐变量编辑。实验结果表明：增加训练数据量能够稳定降低损失并改善生成质量；在本任务中 Dropout 并不适合作为额外正则项；隐变量维度需要在表达能力与优化难度之间折中，20 维表现较好；激活函数差异相对较小，其中 ReLU 获得了略优的损失结果。

总体来看，VAE 能够在较简单的卷积结构下学习 MNIST 数字的连续潜空间，适合用于图像生成和可解释的隐变量编辑。但其生成图像仍存在一定模糊感，后续可尝试改进重建损失、引入  $\beta$ -VAE、条件 VAE 或更强的生成模型结构，以提升生成质量和可控性。

## 8 参考文献

- [1] Kingma D P, Welling M. Auto-Encoding Variational Bayes[C]. International Conference on Learning Representations, 2014.



- [2] LeCun Y, Bottou L, Bengio Y, Haffner P. Gradient-Based Learning Applied to Document Recognition[J]. Proceedings of the IEEE, 1998, 86(11): 2278-2324.
- [3] Paszke A, Gross S, Massa F, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library[C]. NeurIPS, 2019.

## A 完整实验结果截图

本附录按实验组给出结果目录中的全部图片，包括损失曲线、重建结果、随机生成结果与隐变量编辑结果。

### A.1 数据量 10%

仅使用 MNIST 训练集的 10%

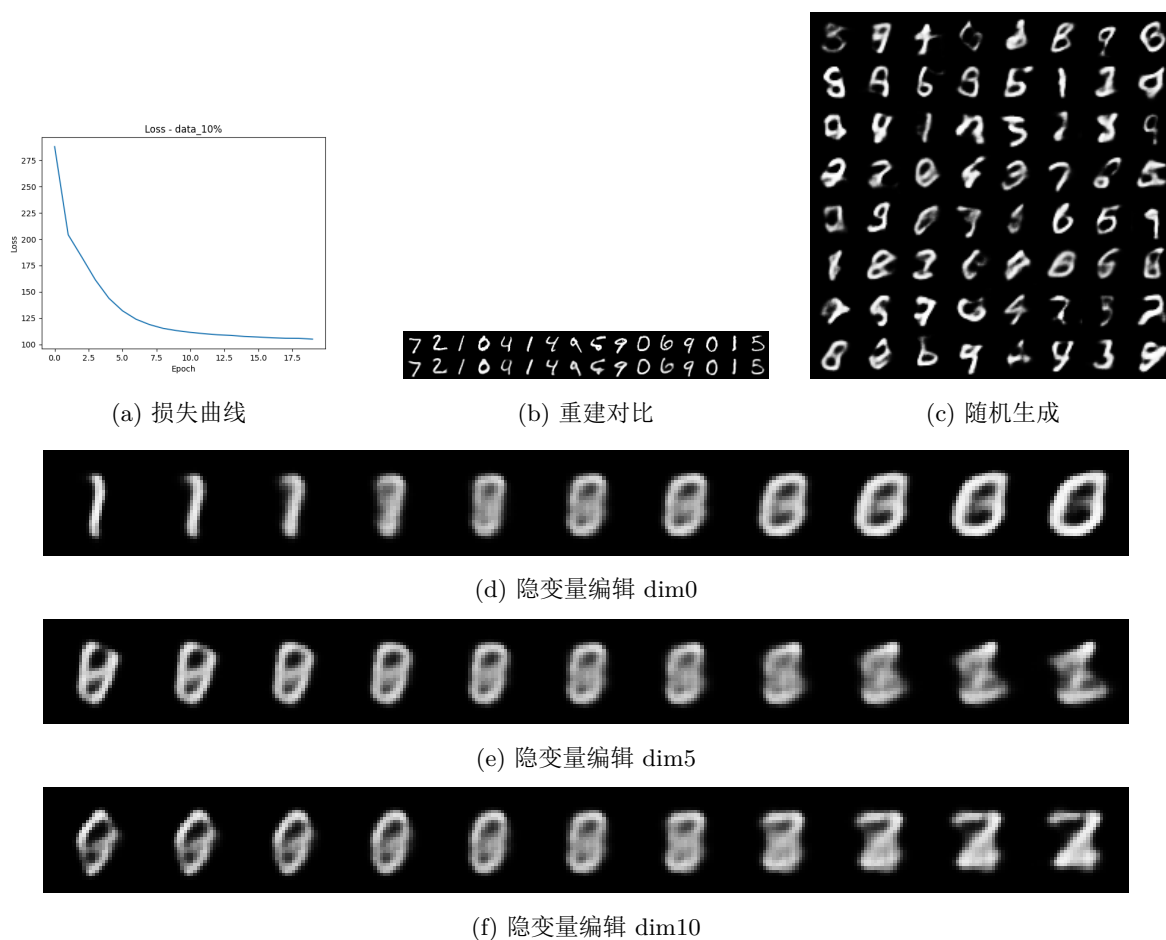


图 3 数据量 10% 的训练曲线、重建、生成与隐变量编辑结果

### A.2 数据量 30%

仅使用 MNIST 训练集的 30%

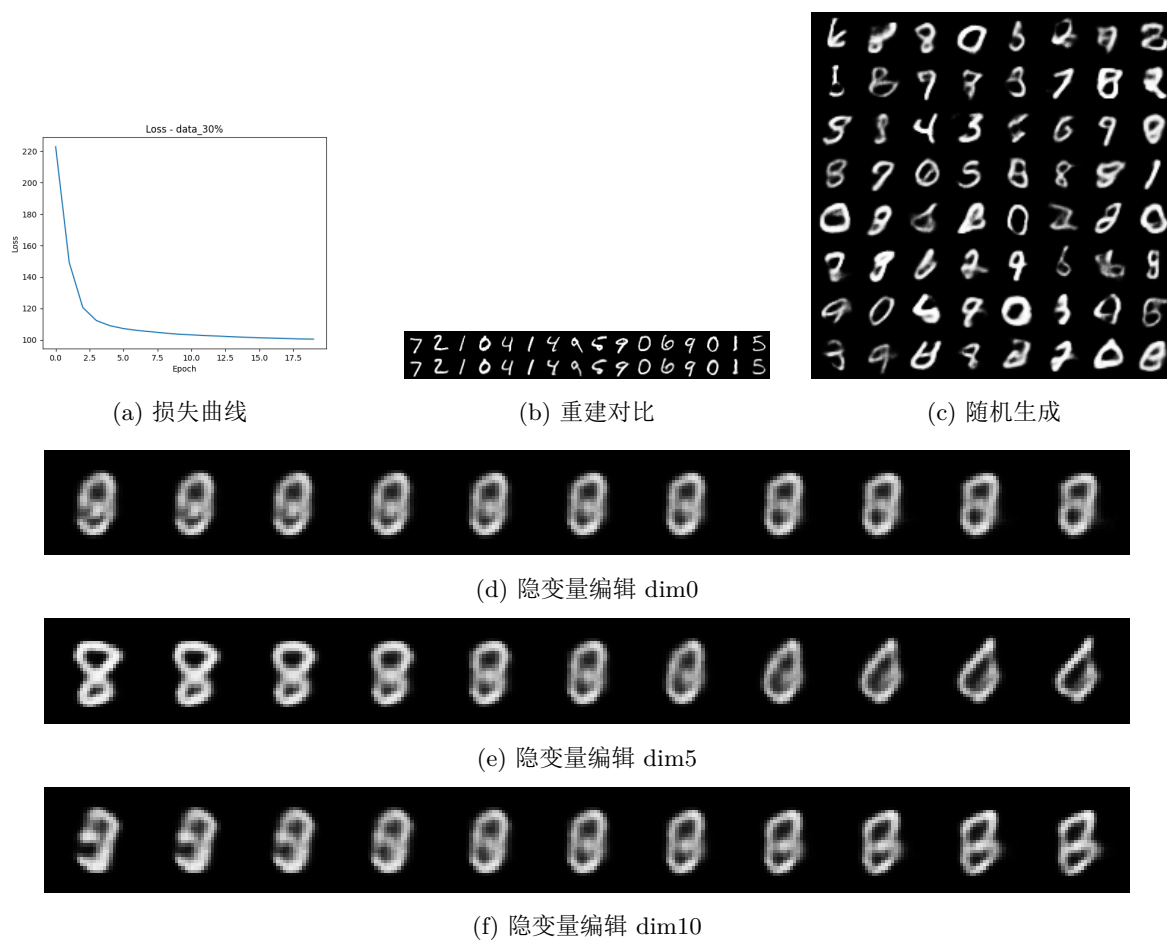


图 4 数据量 30% 的训练曲线、重建、生成与隐变量编辑结果

### A.3 数据量 100%

使用完整 MNIST 训练集

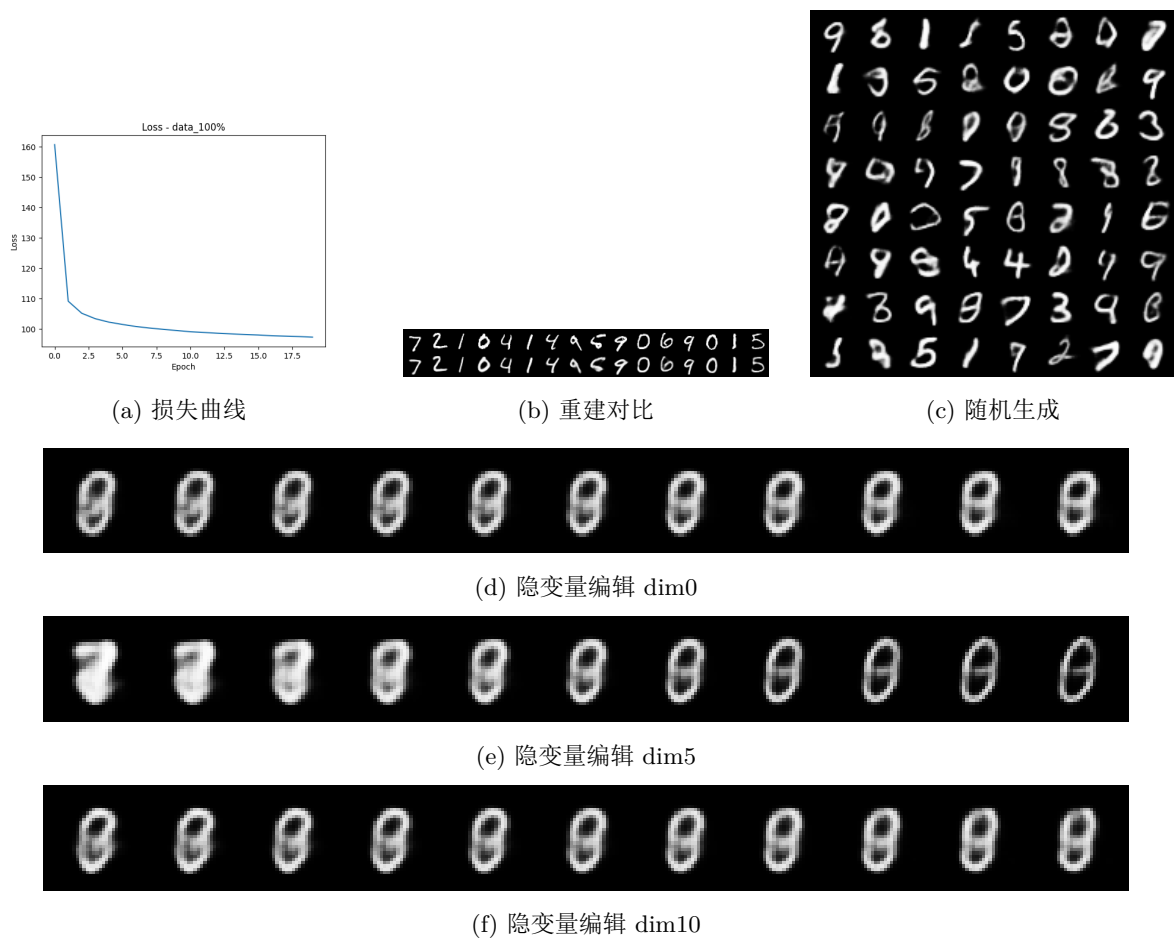


图 5 数据量 100% 的训练曲线、重建、生成与隐变量编辑结果

#### A.4 不使用 Dropout

编码器与解码器均不加入 Dropout

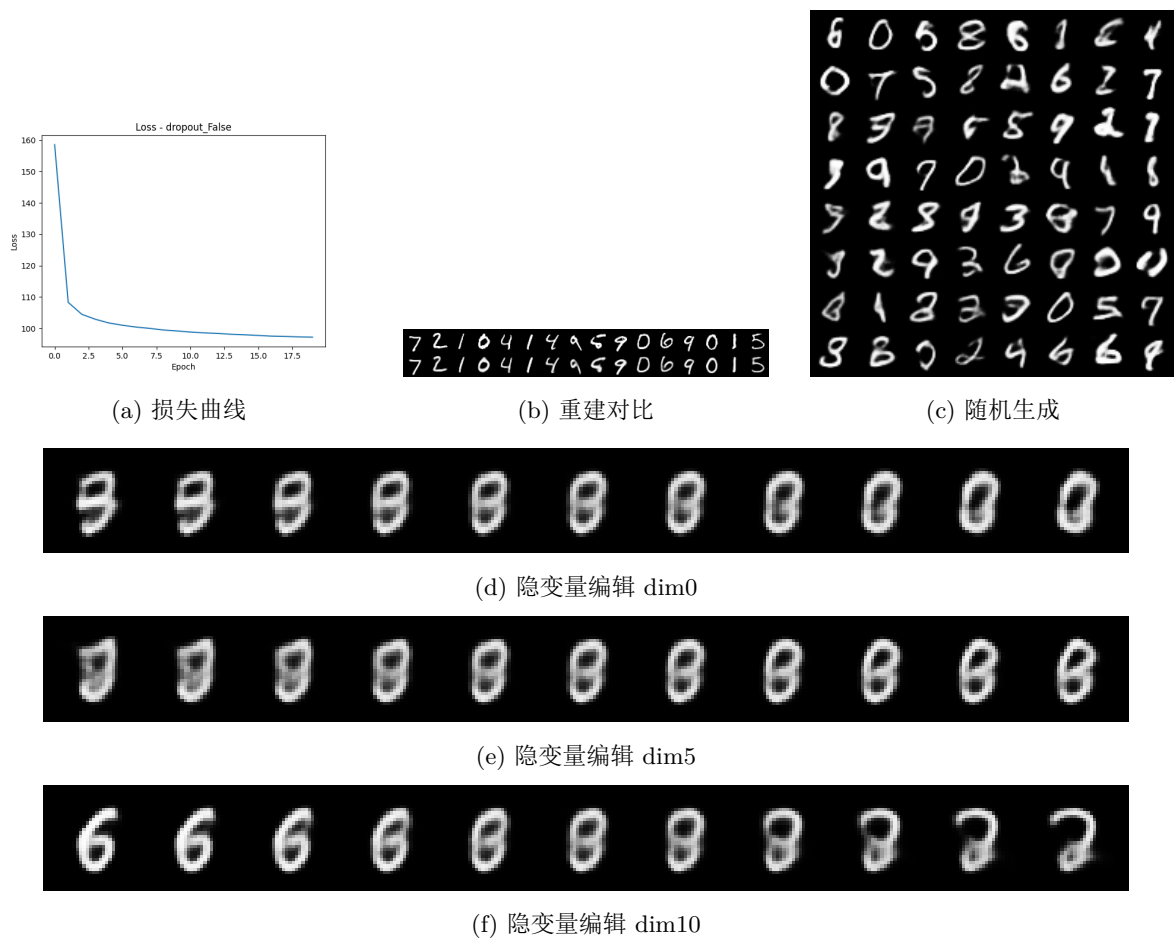


图 6 不使用 Dropout 的训练曲线、重建、生成与隐变量编辑结果

## A.5 使用 Dropout

Dropout 概率为 0.2

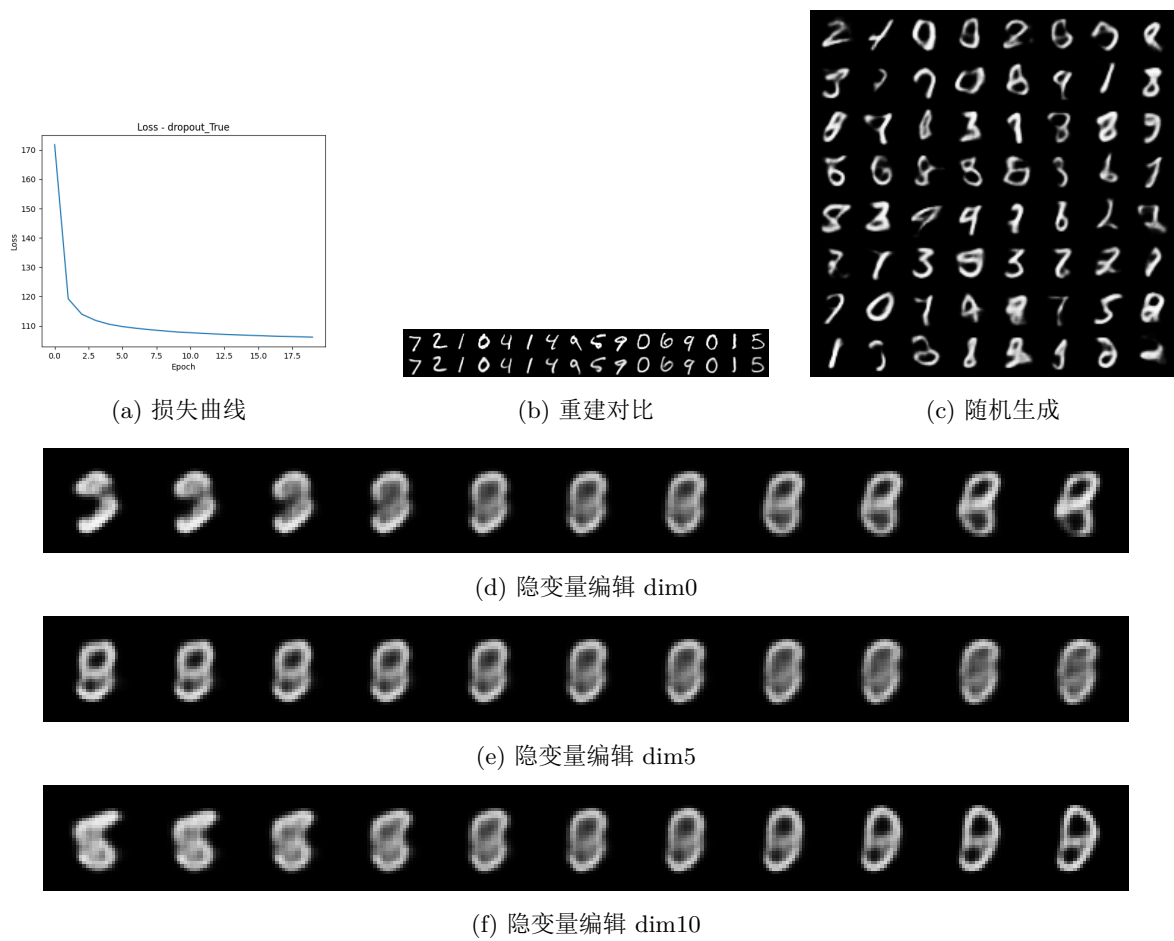


图 7 使用 Dropout 的训练曲线、重建、生成与隐变量编辑结果

## A.6 隐变量维度 10

潜空间维度设为 10

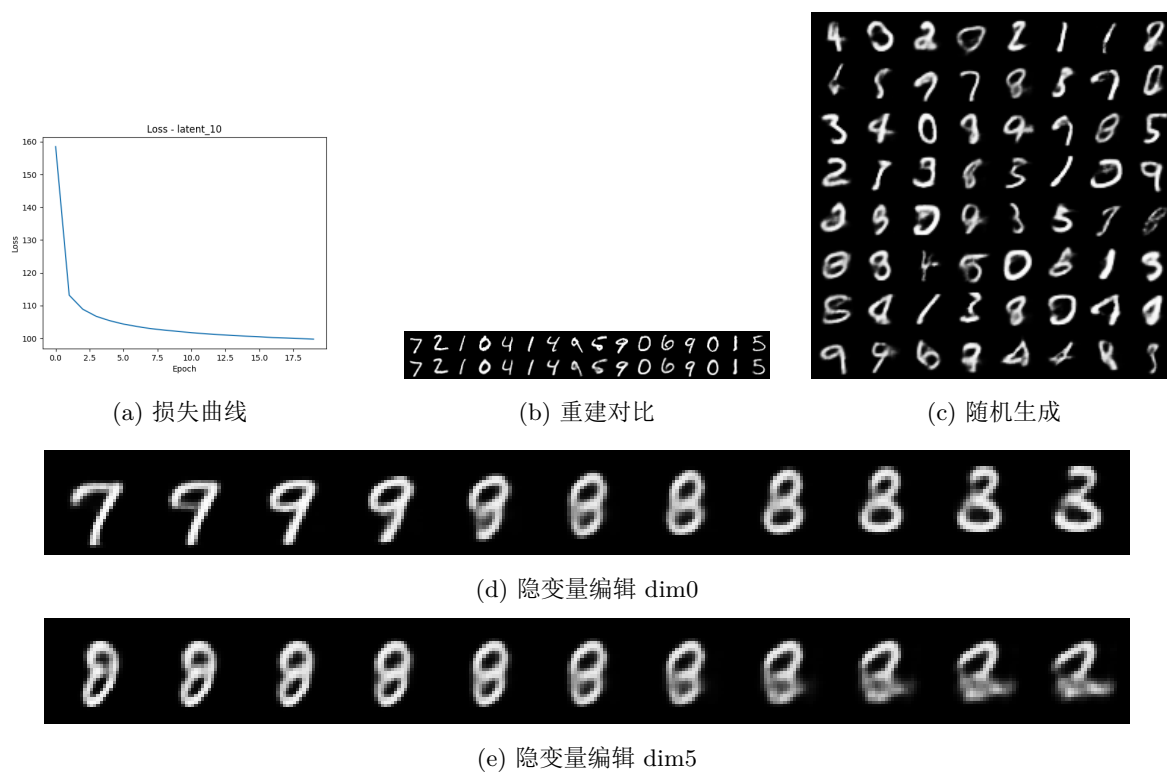


图 8 隐变量维度 10 的训练曲线、重建、生成与隐变量编辑结果

## A.7 隐变量维度 20

潜空间维度设为 20

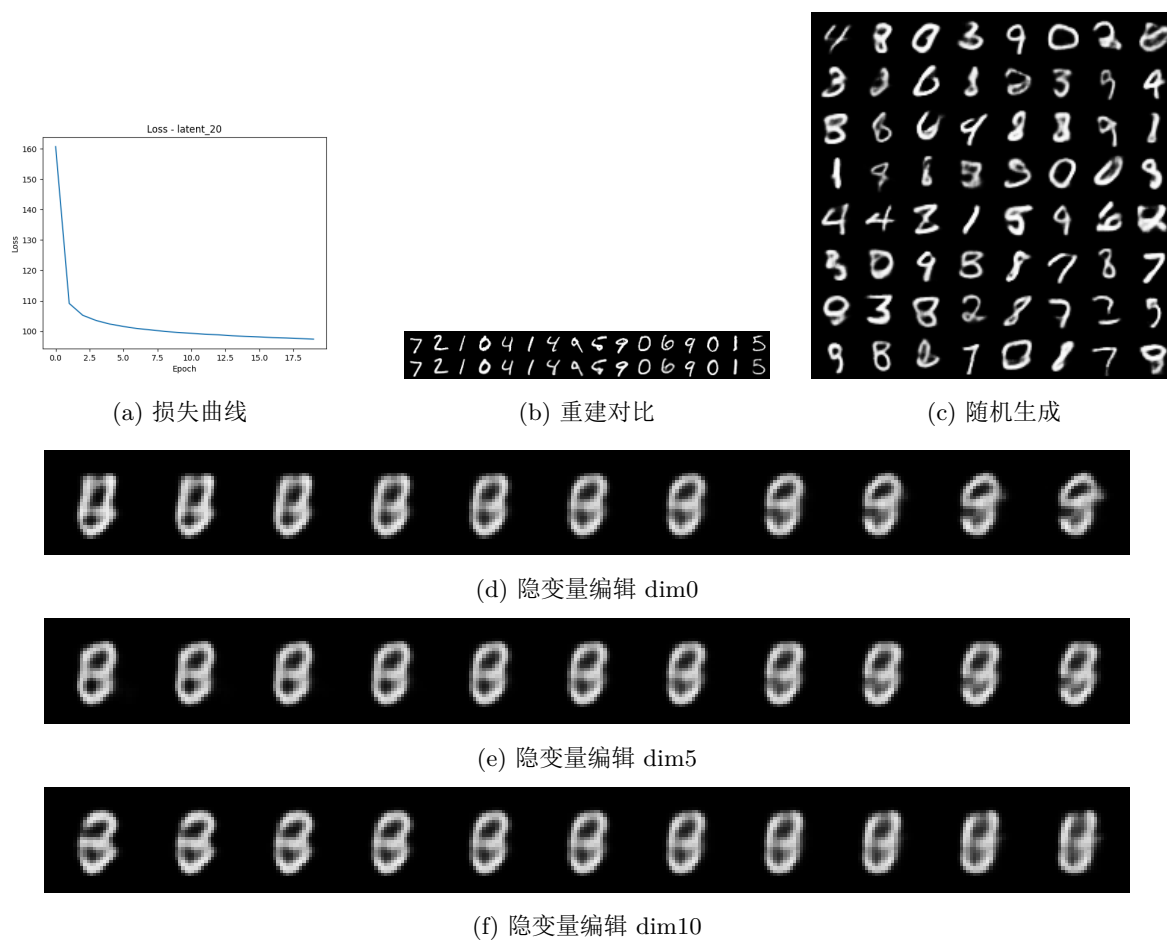


图 9 隐变量维度 20 的训练曲线、重建、生成与隐变量编辑结果

## A.8 隐变量维度 50

潜空间维度设为 50

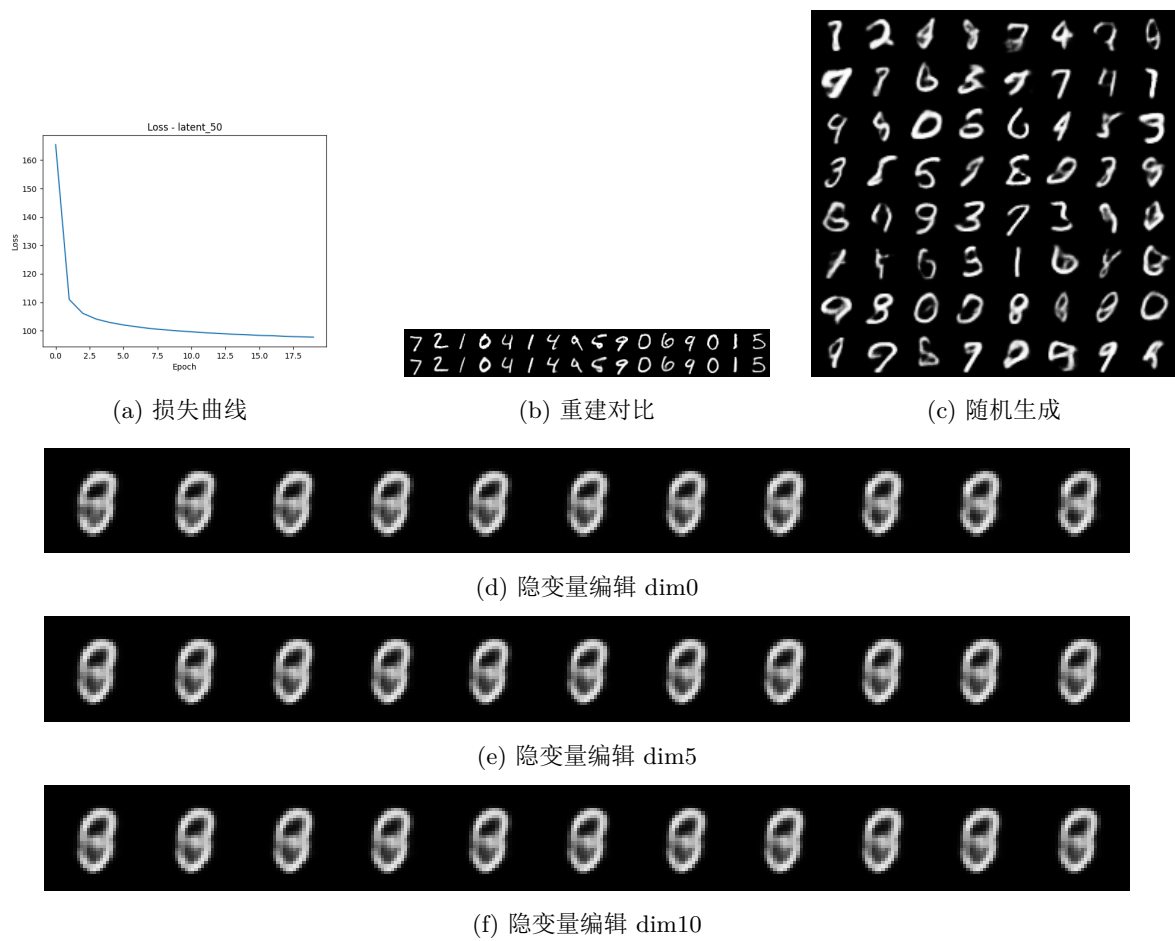


图 10 隐变量维度 50 的训练曲线、重建、生成与隐变量编辑结果

## A.9 ReLU 激活函数

卷积层和反卷积层使用 ReLU



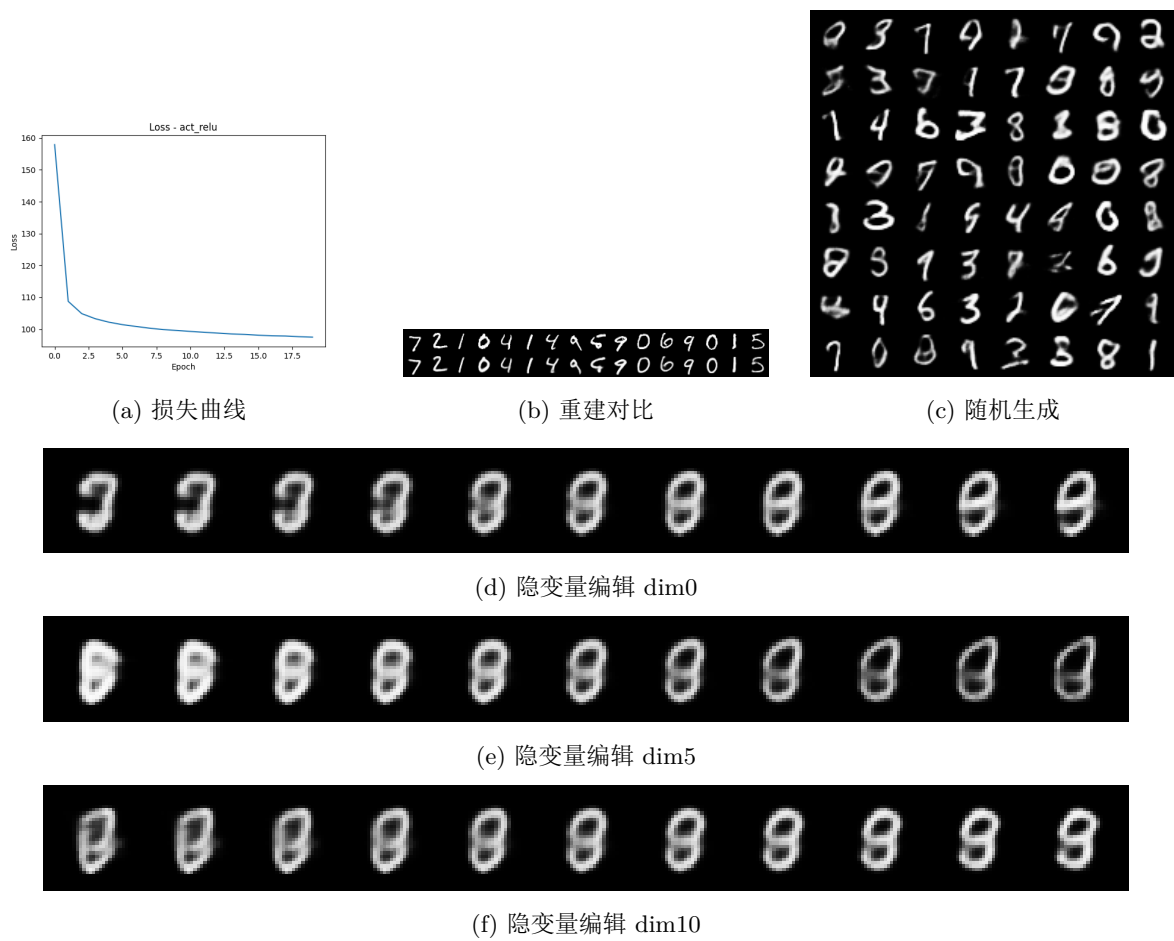


图 11 ReLU 激活函数的训练曲线、重建、生成与隐变量编辑结果

## A.10 Leaky ReLU 激活函数

负半轴斜率为 0.2

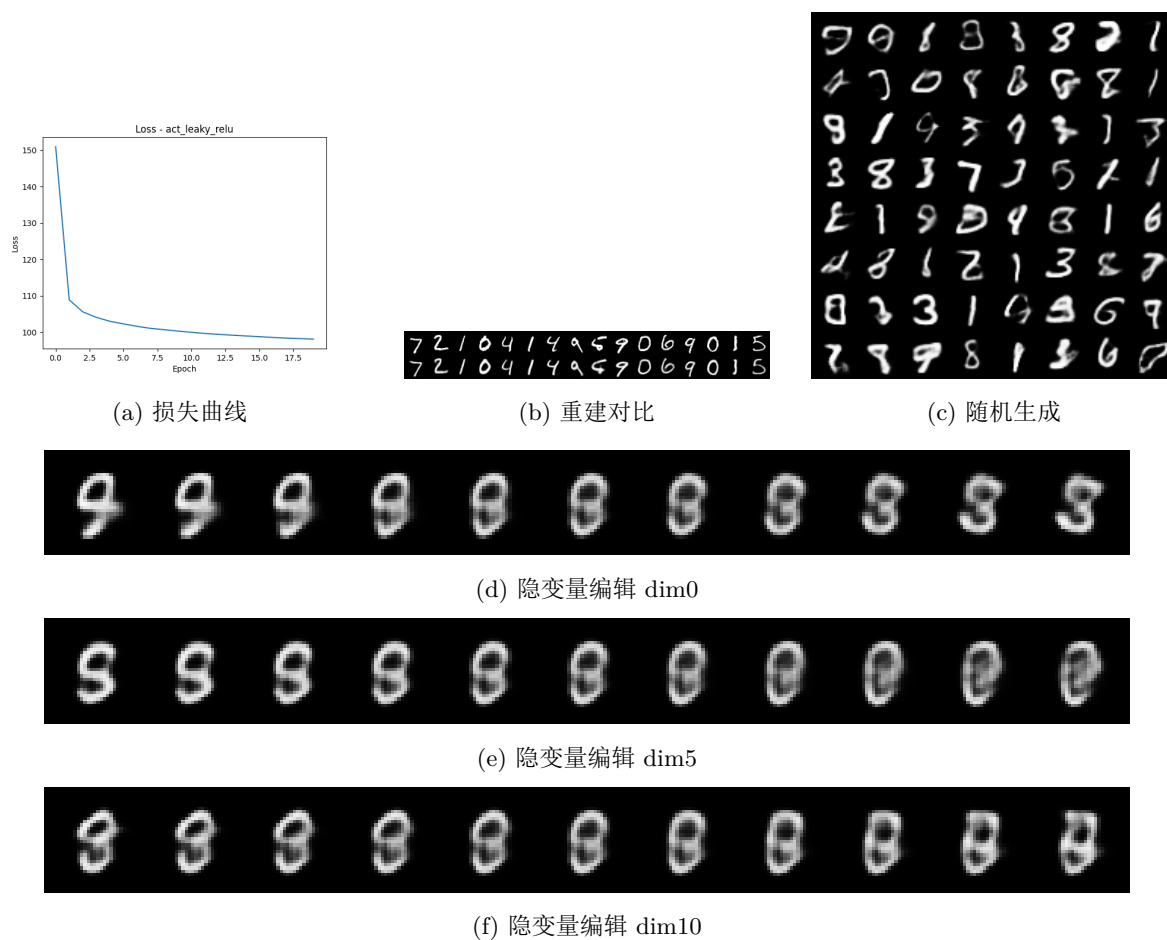
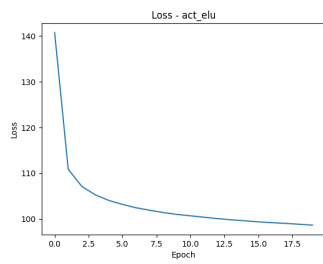


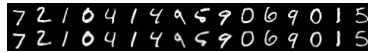
图 12 Leaky ReLU 激活函数的训练曲线、重建、生成与隐变量编辑结果

## A.11 ELU 激活函数

使用 ELU 非线性函数



(a) 损失曲线



(b) 重建对比



(c) 随机生成



(d) 隐变量编辑 dim0



(e) 隐变量编辑 dim5



(f) 隐变量编辑 dim10

图 13 ELU 激活函数的训练曲线、重建、生成与隐变量编辑结果